

A Taxonomy of AI Hallucinations

Eight classes and fifty-one types, found by measuring real work

Johnny Morgan

University of Maryland, Baltimore County

August 28, 2026

Abstract

Most people who have used a generative AI system have heard the word “hallucination” and understand it to mean the model made something up. That is part of it. It is not most of it.

We instrumented an extended body of real work, an AI assistant doing research engineering across hundreds of working sessions, and recorded every failure we could detect and confirm. The result is 51 named failure types in 8 classes, across 1,690 observed events.

Fabrication, the making-things-up class everyone already knows about, accounts for 29% of those events. The two most frequent individual failures are not fabrication at all. The most common is the model choosing a slow method when a fast one was available. The second is rebuilding a tool that already existed.

This document is a short orientation, not a full treatment. It defines what a hallucination is in plain terms, sets out the eight classes, lists all 51 types with one-line definitions, and states plainly what the numbers do and do not support.

1 What a hallucination is

A useful working definition, and the one this taxonomy is built on:

A hallucination is output the model presents with confidence that is not grounded in anything real: not in the data it was given, not in the tools it has, and not in what actually happened.

Three things are worth pulling out of that.

It is about grounding, not about truth. A model can state something true by accident, having checked nothing. That is still the failure we care about, because the process that produced it will produce a falsehood next time under the same conditions.

It is about confidence. The model sounds the same whether it verified a claim or guessed it. If unverified output carried an audible hedge, this would be a much smaller problem. It does not, which is why the failure is invisible exactly where a reader would otherwise catch it.

It is not only about facts. A model that invents a citation has hallucinated. So has a model that says it finished a task it did not finish, that rebuilds a tool it was told already exists, or that summarises its own work as tidier than it was. All four are ungrounded confidence. Only the first matches the popular usage.

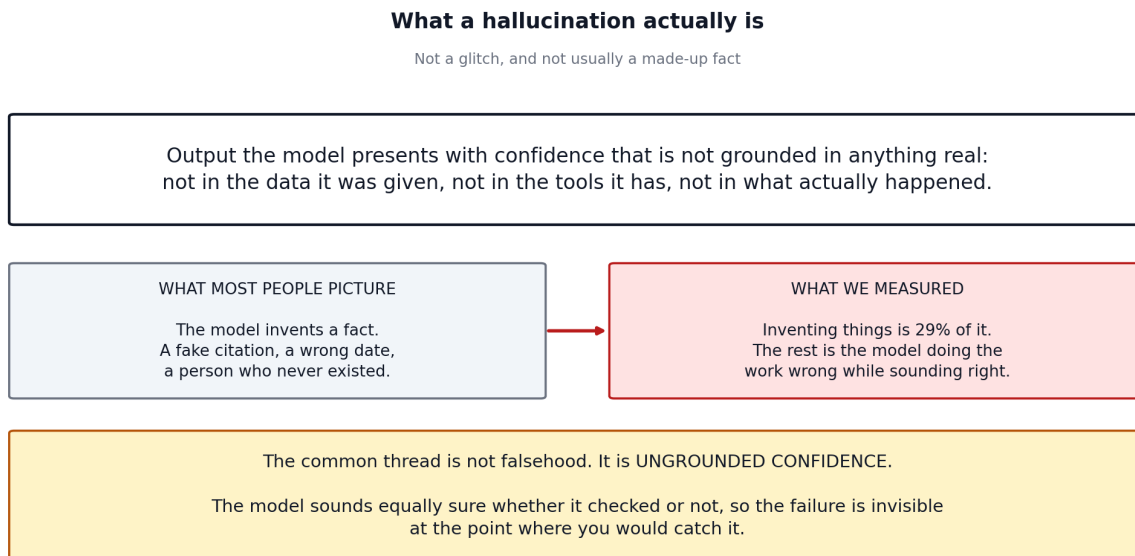


Figure 1: The definition, and the gap between what the word usually means and what we measured.

2 How these were found

The taxonomy was not designed in advance and then looked for. It accumulated the other way around.

An AI assistant worked on a long-running research project across hundreds of sessions. Those sessions were recorded: every instruction, every tool call, every file written, every correction the human made. When something went wrong, it was named, defined, and added to a registry. Detection was then automated where it could be, so the same failure could be counted rather than merely remembered.

Three consequences of that method matter when reading the numbers.

The counts are a floor, not a total. They count failures that were detected. A failure nobody noticed is not in here.

The distribution reflects this kind of work. These are sessions of research engineering: writing code, querying data, deploying jobs. A customer service assistant or a document summariser would show a different mix. The classes are likely to generalise. The proportions are not.

Naming came before measuring, and the gap is visible. Six of the 51 types have been named and defined but not yet observed in the instrumented record. They are kept in the taxonomy because a named failure with no observations is a different thing from a failure that does not happen.

3 The eight classes

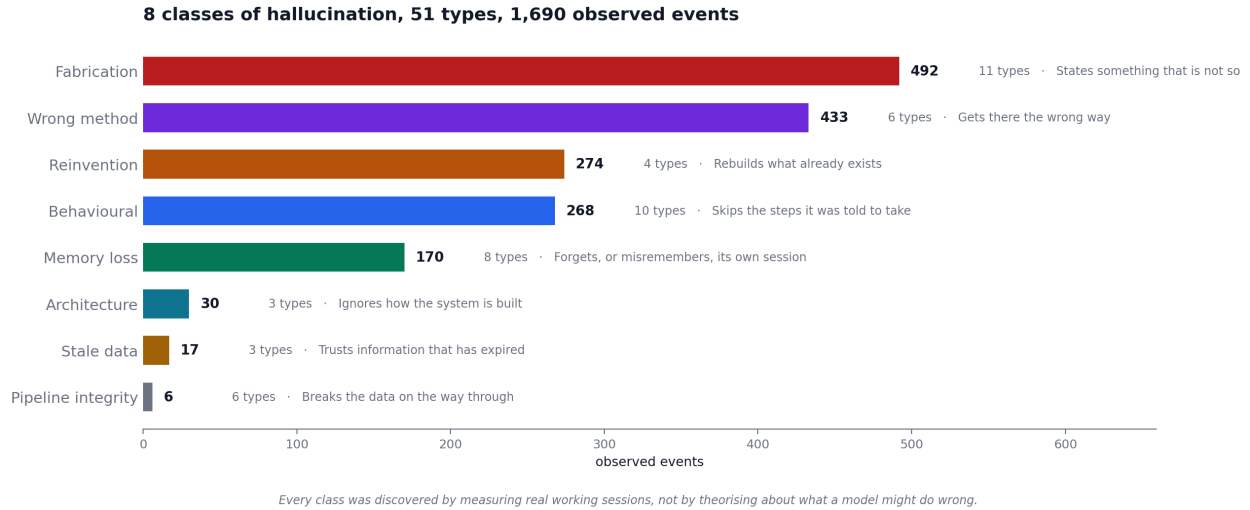


Figure 2: The eight classes, ordered by observed events. Fabrication leads, but it is under a third of the total.

Class	In one line	Types	Events
Fabrication	States something that is not so	11	492
Wrong method	Gets there the wrong way	6	433
Reinvention	Rebuilds what already exists	4	274
Behavioural	Skips the steps it was told to take	10	268
Memory loss	Forgets, or misremembers, its own session	8	170
Architecture	Ignores how the system is built	3	30
Stale data	Trusts information that has expired	3	17
Pipeline integrity	Breaks the data on the way through	6	6
Total		51	1,690

Table 1: The eight classes. “Memory loss” is called `compaction.artifacts` in the data: it covers what happens when a session runs long enough that the model must compress its own history and loses things in the process.

4 What the distribution says

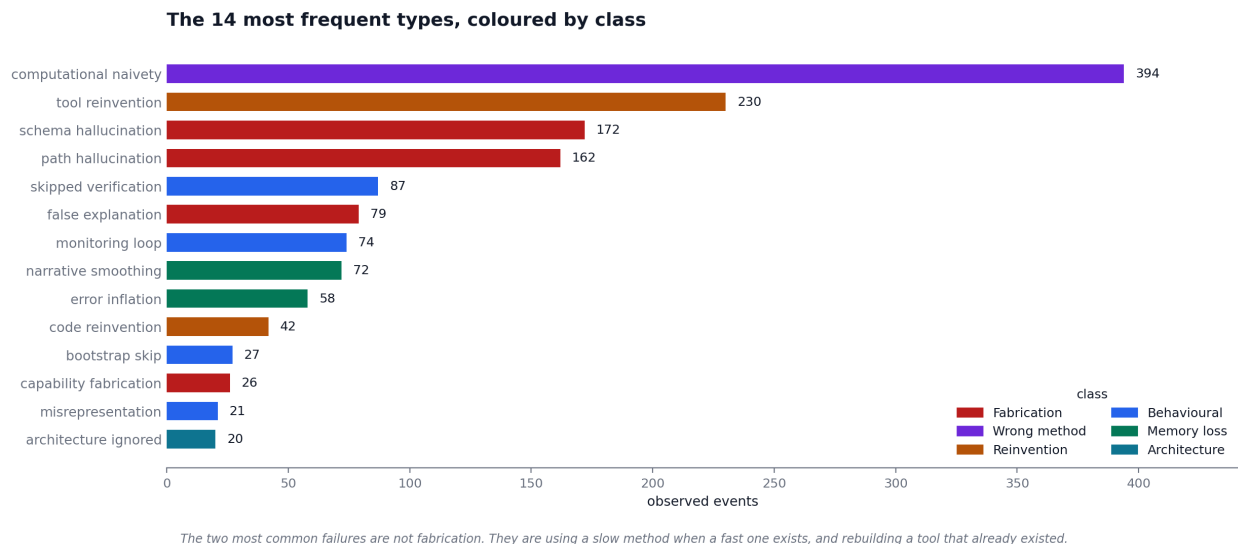


Figure 3: The sixteen most frequent types. The top two are not fabrication.

Fabrication is 29% of events. It is the largest single class and it is still under a third. A programme that addresses only fabrication addresses under a third of the problem, while reporting that it has solved “hallucination”.

The single most common failure is a performance failure. `computational_naivety`, 394 events, is the model writing a slow loop when a fast bulk method was available. Nothing it produced was false. The work was simply done the wrong way, at a cost nobody would see except in the clock.

The second is a duplication failure. `tool_reinvention`, 230 events, is the model writing new code to do a job an existing tool already did. The output usually works. It is still waste, and it still quietly grows the amount of code somebody has to maintain.

Two fabrication types dominate their class, and both are structural. `schema_hallucination` (172 events) is querying a database using column names that do not exist. `path_hallucination` (162) is referring to files that are not there. These are the two most confirmable failures in the whole taxonomy, because a path either exists or it does not. That is why they carry the highest structural confirmation counts, 164 and 155 respectively, and it suggests something more general: the failures we can count best are the ones the world can contradict directly.

5 How far each type has been taken

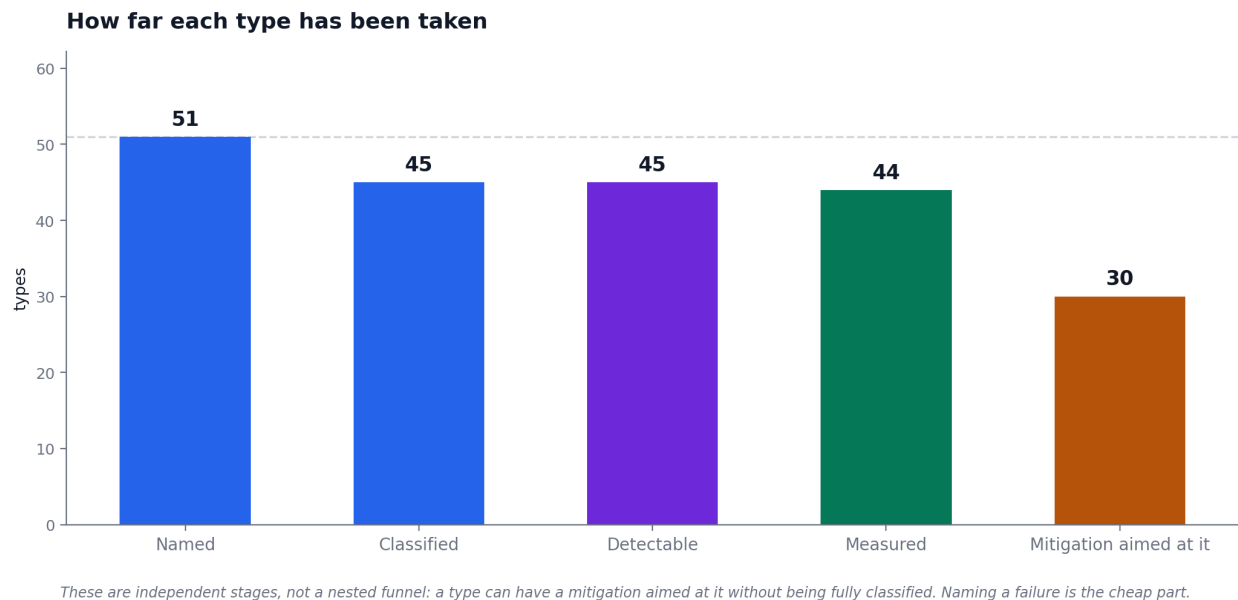


Figure 4: Naming a failure is the cheap part. These are independent stages rather than a nested funnel: a type can have a mitigation aimed at it without being fully classified.

All 51 types are named and defined. 45 are classified in the detection matrix, 44 have had their performance measured over time, and 30 have some mitigation aimed specifically at them.

The gap between naming and mitigating is where the work actually stands. It is straightforward to notice a failure and write down what it was. It is harder to detect it automatically, harder again to design something that reduces it, and hardest of all to show that the reduction is real rather than a lull.

6 The full list

Every type, grouped by class, ordered by observed events within each class.

Type	Events	What it is
FABRICATION 11 types, 492 events		
schema_hallucination	172	Queries a database using column or table names that do not exist
path_hallucination	162	References file paths that do not exist, or writes to the wrong place
false_explanation	79	Gives a confident causal explanation with no evidence behind it
capability_fabrication	26	Claims a feature, tool, or API exists when it does not
capability_overstatement	13	Plans actions it cannot actually perform in this environment
fabricated_fact	9	Asserts a specific fact with no basis in the code or the runtime
gpu_config_hallucination	8	Misstates the hardware configuration. The most-repeated single error here

Type	Events	What it is
attribution_misframing	8	Frames a structural gap as a personal oversight, implying agency it lacks
recurring_false_belief	7	Repeats a false claim after being corrected
defensive_fabrication	7	Invents a constraint to avoid admitting it must backtrack
rationalized_observation	1	Explains an accidental state as though it were deliberate design
WRONG METHOD 6 types, 433 events		
computational_naivety	394	Uses a slow loop when a fast bulk method exists
redundant_recomputation	17	Recomputes from scratch when the answer was already cached
deployment_sequence_error	8	Runs deployment steps out of order, or mishandles a stuck state
agent_prompt_underspecification	6	Tells a sub-agent the task but not the context, so it falls back to defaults
serial_when_parallel	5	Runs on one core or one GPU when many were available
search_scope_narrowing	3	Searches where it expects things to be and misses the obvious place
REINVENTION 4 types, 274 events		
tool_reinvention	230	Ignores an available tool and writes ad-hoc code instead
code_reinvention	42	Writes new code for something the codebase already does
foundational_rederivation	2	Rebuilds an authoritative artifact from scratch rather than fixing one row
infrastructure_reinvention	0	Rebuilds infrastructure that already exists
BEHAVIOURAL 10 types, 268 events		
skipped_verification	87	Proceeds to full execution without checking intermediate results
monitoring_loop	74	Blocks the session polling or sleeping in a loop
bootstrap_skip	27	Starts work without the mandatory session setup, cascading failures later
misrepresentation	21	Says work is done or working when it is partial or untested
unwarranted_confidence	17	Asserts a fact without verifying it, and is wrong
unsolicited_development	13	Builds a full implementation without asking first
scope_inflation	13	Expands the job beyond what was asked
exuberance	6	Writes far more code than requested
tool_preference_persistence	6	Reverts to a familiar tool after being corrected three or more times
cross_session_amnesia	4	Repeats an approach that already failed in an earlier session
MEMORY LOSS 8 types, 170 events		
narrative_smoothing	72	Summarises its own session as tidier and more sequential than it was
error_inflation	58	Misreports successful operations as errors when summarising
compression_artifact	14	Compresses distinct items into “repeated” or “similar”, losing the distinction
post_compaction_amnesia	9	Keeps the gist after compressing history but loses the operational detail
message_dropping	9	Omits a user instruction entirely from its summary
attribute_substitution	6	Identifies the right thing but attaches the wrong properties to it

Type	Events	What it is
context_rotation_drift	2	Drifts further from the original intent with each successive summary
attribution_drift	0	Attaches a real fact to the wrong entity
ARCHITECTURE 3 types, 30 events		
architecture_ignored	20	Knew the established pattern and violated it anyway
architecture_unknown	7	Designed something without knowing the existing architecture
architecture_reinvented	3	Duplicated an existing design decision one level up
STALE DATA 3 types, 17 events		
stale_ephemeral_state	10	Assumed a process or hardware state was still current
stale_data_citation	4	Cited an out-of-date source as though it were current
stale_data_no_freshness	3	Did not check how old the data was before citing it
PIPELINE INTEGRITY 6 types, 6 events		
index_misalignment	5	Joined data sources that use incompatible identifiers
label_miscount	1	Assumed the wrong number of classes in a dataset
truncation_loss	0	An infrastructure bug silently destroyed content
circular_validation	0	A check consumes the labels it was supposed to verify, so it always passes
dedup_collision	0	A key design error silently dropped rows during deduplication
stitched_spaces	0	Combined outputs from models that are not comparable

7 What this does not show

One project, one kind of work. These sessions are research engineering. The proportions would look different for a chatbot, a summariser, or a coding assistant used a different way. Treat the classes as portable and the percentages as local.

Detected, not total. Every count is a floor. The types with the highest confirmation counts are the ones reality can contradict most directly, which means the taxonomy is better at counting checkable failures than subtle ones. A low number can mean a rare failure or a hard-to-see one, and this table does not distinguish them.

Counts depend on which record you use. The figure of 1,690 is the observed-event count in the detection matrix. Other counts in the same system answer slightly different questions and give different totals. Any single number quoted from this work should name the record it came from.

Two internal records disagree on the type list. A curated registry holds 31 types; the inventory used here holds 51. The 20-type difference includes the two largest fabrication types. This document uses the inventory. Reconciling the two is open work.

8 Where a deeper discussion would go

Four questions this orientation raises and does not answer.

1. **Does naming a failure reduce it?** The record includes the dates a type was named and the dates mitigations were introduced, so this is measurable rather than a matter of opinion.

Some types were suppressed after mitigation. Others recurred regardless.

2. **Which failures are worth mitigating?** A type with 394 events that costs a few seconds each may matter less than a type with 7 events that silently corrupts a result.
3. **Can a model detect its own hallucinations in flight?** Detecting them after the fact in a transcript is a solved problem for several of these types. Catching them at the moment of production is not.
4. **How much transfers?** The eight classes come from one project. Whether they cover generative AI failure generally is an empirical question nobody has run.